

KULLIYAH OF INFORMATION TECHNOLOGY AND COMMUNICATION

INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA (IIUM)

WEB APPLICATION DEVELOPMENT (BIIT 2305)

SEMESTER II, 2025/2026

SECTION 5

GROUP ASSIGNMENT

TOPIC:

FOODSAVER: MANAGEMENT WEB APPLICATION FOR FOOD LEFTOVERS

INSTRUCTOR: DR. NAJHAN BIN MUHAMAD IBRAHIM

GITHUB REPOSITORY:

<https://github.com/hwhwbw-root/food-management>

README:

<https://github.com/hwhwbw-root/food-management/blob/main/README.md>

WEBSITE LINK:

<https://food-management-production.up.railway.app/>

NAME	MATRIC NUMBER
AZIZUL HAKIM BIN AWANG ABDUL RAHIM	2419087
MUHAMMAD FARIHIN BIN JOHAR	2410499
NADHIRATUL INSYIRAH BINTI ESMADI	2411198
NAUFA BINTI MOHD YAZID	2413304

TABLE OF CONTENT

1.0 EXECUTIVE SUMMARY	3
1.1 Project Overview	3
1.2 Objectives Achieved	3
2.0 PROBLEM STATEMENT	4
2.1 Problem Background	4
2.2 Problem Statement	4
2.3 Project Objectives	5
2.4 Project Scope	5
3.0 SYSTEM DESIGN	7
3.1 Entity Relationship Diagram (ERD)	7
3.2 System Sequence Diagram	8
3.3 System Architecture Overview	10
4.0 TECHNICAL IMPLEMENTATION	11
4.1 Models & Database Migrations	11
4.2 Routes Configuration	13
4.3 Controllers & CRUD Logic	in 14
4.4 User Authentication & Security	15
4.5 Views & Blade Template Engine	16
5.0 USER INTERFACE DESIGN	18
5.1 Use of Media (20 marks)	18
5.2 Design, Colour Scheme & Layout (20 marks)	19
5.3 Navigation & Links (10 marks)	20
5.4 UI Walkthrough (Page by Page)	21
6.0 SYSTEM TESTING	27
6.1 Test Cases Table	27
6.2 Deployment Verification	28
7.0 CHALLENGES AND SOLUTIONS	30
7.1 Technical Challenges	30
7.2 Team & Time Constraints	31
8.0 CONCLUSION	32
8.1 Summary of Achievements	32
8.2 Future Improvements	32
REFERENCES	33

1.0 EXECUTIVE SUMMARY

1.1 Project Overview

FoodSaver is a centralized web application developed using the Laravel Model-View-Controller (MVC) framework. The main purpose of the system is to help reduce food waste in Malaysia by connecting food providers with consumers through a real-time digital platform. Food businesses such as restaurants, hotels, home bakers, and catering services can use the platform to share surplus food with people who are looking for affordable or free food options.

The system was built using Laravel 10.x (PHP 8.1+), MySQL for database management, the Blade templating engine for dynamic web pages, and Bootstrap 5 to ensure a responsive and user-friendly interface.

Instead of relying on informal methods such as social media posts or word-of-mouth communication, FoodSaver provides a more organized and reliable platform for food redistribution. The system benefits both vendors and buyers while supporting Islamic values by encouraging responsible consumption, reducing waste (Israf), and promoting community welfare.

1.2 Objectives Achieved

The project successfully achieved its main objectives:

- Developed a complete food surplus management system using the Laravel MVC framework, allowing vendors to create, view, update, and delete food listings through a full CRUD implementation.
- Implemented a secure authentication system with two user roles: Vendors and Buyers. Each role has its own dashboard and access permissions based on their responsibilities.
- Enables buyers to view available surplus food listings in real time, making it easier to find food opportunities without depending on social media or informal communication channels.
- Streamlined the food redistribution process by allowing vendors to post surplus food while buyers can browse, reserve, and claim available items through the platform.
- Successfully deployed the system on Railway and completed the final documentation covering the system's design, implementation process, and key functionalities.

2.0 PROBLEM STATEMENT

2.1 Problem Background

Food waste in Malaysia has become a serious issue that requires immediate attention from both the public and private sectors. Based on data from the Solid Waste Management and Public Cleansing Corporation (SWCorp), around 17,000 tonnes of food waste are generated every day nationwide. What is more concerning is that about 4,005 tonnes of this food are still edible, which could potentially feed around 2.9 million people with three meals a day.

At the same time, there is a large amount of surplus food produced by food-related businesses such as restaurants, hotels, and catering services. However, there is currently no proper centralized system that effectively connects these food providers with people who need affordable or free food. As a result, both sides face difficulties including the businesses struggle to redistribute excess food, while consumers miss out on opportunities to access it.

2.2 Problem Statement

Food surplus management in Malaysia is still largely unstructured and inefficient. There is no dedicated platform that allows food vendors to easily and reliably share their excess food. Because of this, many people depend on informal channels such as social media posts or word-of-mouth, which are often inconsistent and not updated in real time.

This situation leads to several key issues:

- **Food wastage is still widespread** – Without a proper redistribution system, large amounts of edible food are thrown away every day, contributing to environmental damage, economic loss, and social inequality.
- **Poor information access** – Consumers have no centralized or real-time source to check available surplus food nearby, which causes many opportunities for food redistribution to be missed.
- **Low operational efficiency** – Many food businesses do not have proper digital tools to manage, track, or distribute their surplus food effectively, making the whole process unorganized and inefficient.

2.3 Project Objectives

The main goal of this project is to develop a centralized web-based platform that connects food vendors with consumers in order to help reduce food waste in Malaysia. To achieve this, several specific objectives have been identified:

1. To develop a functional food surplus management system using the Laravel MVC framework, including full CRUD features that allow vendors to create, update, view, and delete food listings in real time.
2. To implement a secure authentication system that separates users into two roles, namely Vendors and Buyers, where each role has its own dashboard and access permissions.
3. To enable buyers to view available surplus food listings in real time from nearby businesses, replacing less reliable methods such as word-of-mouth and social media updates.
4. To streamline and automate the food redistribution process by allowing vendors to post surplus food while buyers can easily browse, reserve, and claim available listings through a structured system.
5. To prepare a complete final report that documents the system design, implementation process, database structure, and overall system functionality.

2.4 Project Scope

The FoodSaver application is designed to include several core features within its current development scope.

- **User Registration and Authentication:** Users, including both vendors and buyers, are able to register, log in, and manage their accounts securely through Laravel's built-in authentication system.
- **Food Listing Management (CRUD):** Vendors are able to manage food surplus listings by creating, viewing, updating, and deleting them. Each listing includes important details such as food name, quantity, price (including free items), expiry time, and pickup location.
- **Buyer Browsing and Reservation:** Buyers can browse all available food listings, apply filters based on category or location, and reserve items directly through the platform.
- **Role-Based Access Control (RBAC):** The system separates users into two roles, Vendor and Buyer, ensuring that each user only has access to features relevant to their role.

At the same time, certain features are not included in this version of the system. These include payment gateway integration, mobile application development, and delivery logistics management. These functionalities are considered future improvements and are discussed further in Section 8.2.

3.0 SYSTEM DESIGN

3.1 Entity Relationship Diagram (ERD)

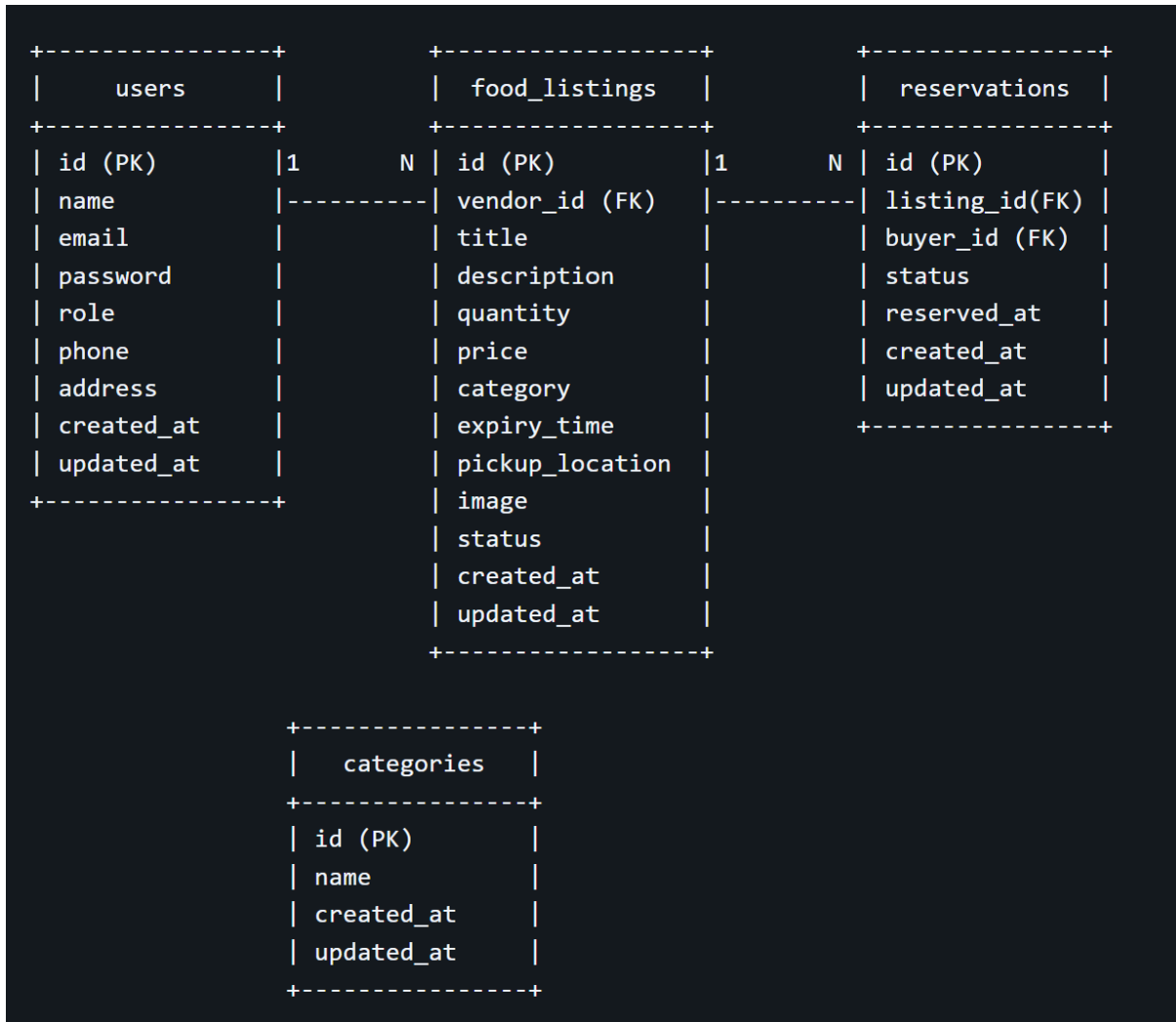


Diagram 3.1 - ERD Diagram

The FoodSaver database structure includes four main tables: users, food_listings, reservations, and categories. Each table is designed to store specific data relevant to the platform’s main functions, and they are linked via foreign key relationships to ensure data integrity throughout the system.

The users table acts as the core of the platform, storing essential account information including contact information, credentials, and user roles (Vendor or Buyer), acting as the main reference for all other entities. The food_listings table manages inventory by monitoring surplus food items; it

maintains a one-to-many relationship with the users table through a vendor_id, allowing a single vendor to create and publish multiple listings featuring detailed metadata and status updates. The reservations table serves as the transaction link, connecting specific buyers to their chosen food listings through buyer_id and listing_id foreign keys, to track the history and status of each reservation effectively. Finally, the categories table acts as a specialized lookup repository, providing a standardized classification framework that allows users to easily filter and explore food items.

To summarize, the key relationships within the database are as follows:

- A User (vendor) can have many Food Listings (one-to-many)
- A Food Listing can have many Reservations (one-to-many)
- A User (buyer) can make many Reservations (one-to-many)
- A Food Listing belongs to one Category (many-to-one)

3.2 System Sequence Diagram

The System Sequence Diagrams illustrate the interaction flow between the user, browser, Laravel Router, Controller, and Database for the two main processes in FoodSaver: a Vendor posting a food listing, and a Buyer browsing and reserving food.

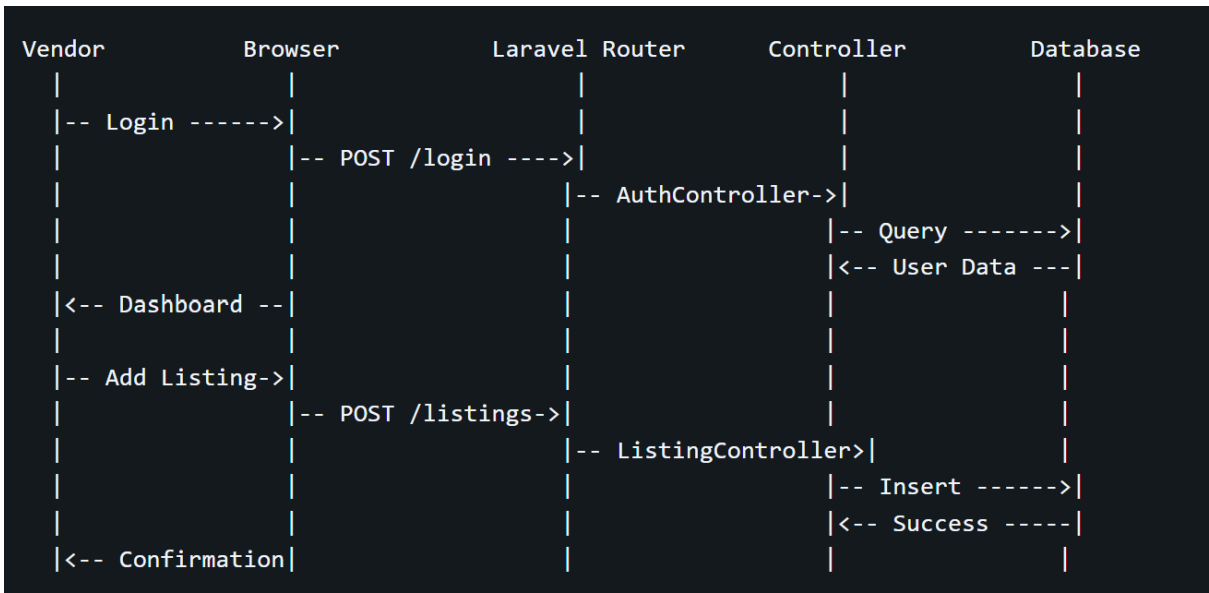
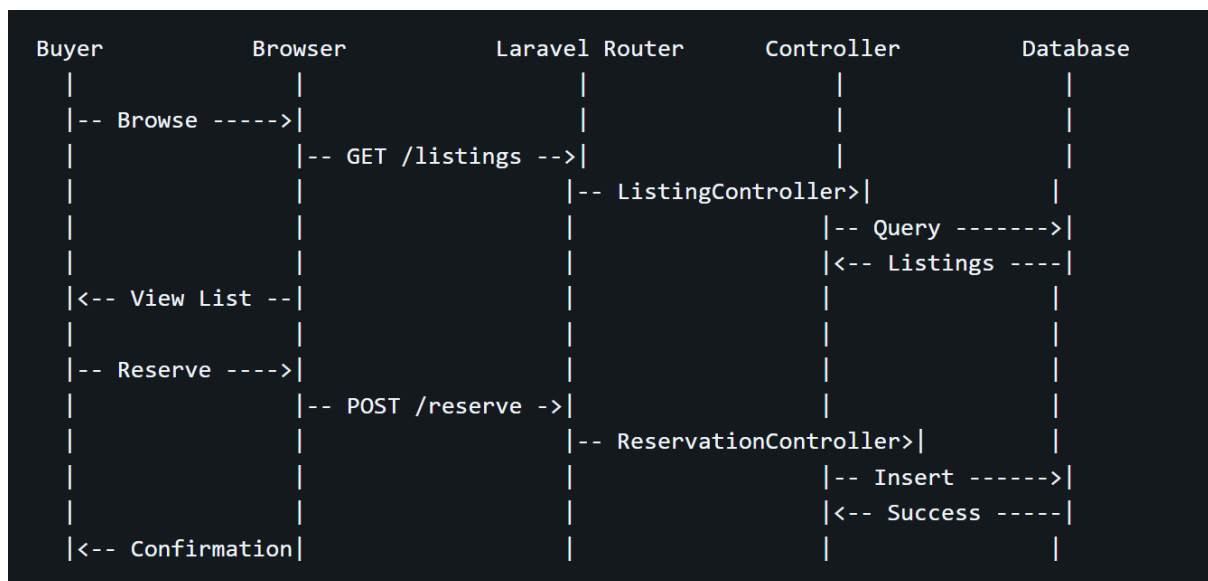


Diagram 3.2 - Vendor (Post a Food Listing)

The diagram depicts the interaction process when a Vendor creates a new food listing on the platform. The process starts when the Vendor enters their login information through the browser, which sends a POST /login request to the Laravel Router. The router then directs this to the AuthController, which checks the database to confirm the user's credentials and returns the corresponding user data. After successful authentication, the Vendor is sent to their dashboard. The Vendor then completes the food listing form and submits it, initiating a POST /listings request. The ListingController receives this from the Laravel Router, handles the input, adds a new entry to the database, and sends back a success response. The Vendor then receives a confirmation indicating that the listing has been successfully established.

Buyer – Browse and Reserve Food



This diagram shows the interaction flow when a Buyer browses available food listings and makes a reservation. The Buyer navigates to the listings page, prompting the browser to issue a GET /listings request to the Laravel Router, which directs it to the ListingController. The controller queries the database for all available listings and returns the results, which are then rendered as a list for the Buyer to view. When the Buyer selects a listing to reserve, the browser sends a POST /reserve request to the Laravel Router, which forwards it to the ReservationController. The controller adds a new

reservation entry to the database and sends a success response, after which the Buyer receives a confirmation of their reservation.

3.3 System Architecture Overview

FoodSaver follows the standard Laravel MVC (Model-View-Controller) framework. A browser request first reaches the Route definitions in routes/web.php, which forwards the request to the suitable Controller. The controller communicates with Eloquent Models to retrieve or store data in the MySQL Database. The processed information is subsequently sent to a Blade View that generates the final HTML response to the browser. Middleware layers sit between the route and controller to implement authentication (auth) and role-based access (role:vendor, role:buyer, role:admin) before any controller logic executes.

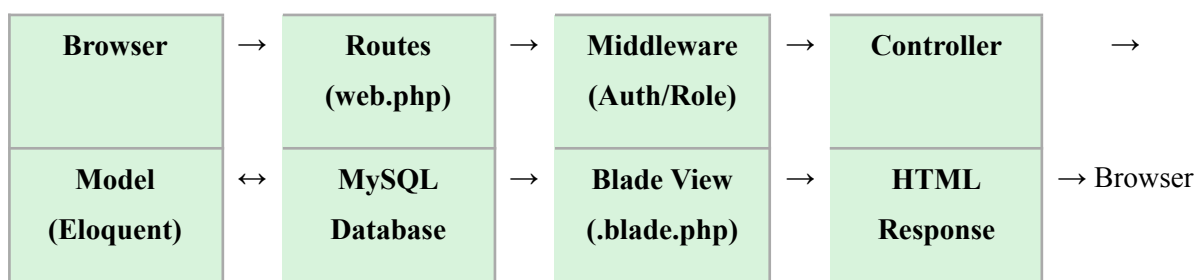


Diagram 3.3 - System Architecture table

4.0 TECHNICAL IMPLEMENTATION

4.1 Models & Database Migrations

The FoodSaver platform is structured around four primary Eloquent models: User, FoodListing, Reservation, and Category. The User model serves as the base, extending Authenticatable and incorporating the HasApiTokens, HasFactory, and Notifiable traits. It is designed to store important profile details such as name, email, password, phone number, address, and a role restricted to 'vendor', 'buyer', or 'admin'. To simplify access control, the model features three helper methods: isVendor(), isBuyer(), and isAdmin(), and maintains two hasMany relationships: one linking vendors to their FoodListing records and another connecting buyers to their Reservation history.

The FoodListing model acts as the central hub for redistribution inventory. It monitors essential item information, such as related supplier, category, title, description, quantity, price, pickup site, image, and the current status of the listing, which is controlled through an enum status of 'available', 'reserved', 'claimed', or 'expired'. The expiry_time is explicitly cast to a datetime format. This model maintains a belongsTo connection to both the User and Category models, and it also possesses a hasMany association with the Reservation model to monitor interest in particular items.

Finally, the Reservation model functions as the link between a buyer and a food listing. It records the specific listing and buyer involved, the current status of the transaction, indicated by an enum of 'pending', 'confirmed', 'cancelled', or 'completed', and the reservation timestamp, which is set to the current time by default. This model establishes two belongsTo relationships, enabling it to conveniently link the particular listing and the buyer related to every reservation. A supporting Category model completes the architecture, delivering the required classification for items available on the platform.

Eloquent Relationships Summary

Model	Relationship	Target
User (vendor)	hasMany	FoodListing (via vendor_id)
User (buyer)	hasMany	Reservation (via buyer_id)

FoodListing	belongsTo	User (vendor)
FoodListing	belongsTo	Category
FoodListing	hasMany	Reservation
Reservation	belongsTo	FoodListing (via listing_id)
Reservation	belongsTo	User (buyer via buyer_id)

Migration Snippet - food_listings table

The following migration defines the food_listings table structure:

```
Schema::create('food_listings', function (Blueprint $table) {
    $table->id();
    $table->foreignId('vendor_id')->constrained('users')->onDelete('cascade');
    $table->foreignId('category_id')->nullable()->constrained('categories')->nullOnDelete();
    $table->string('title');
    $table->text('description')->nullable();
    $table->unsignedInteger('quantity');
    $table->decimal('price', 8, 2)->default(0);
    $table->string('pickup_location');
    $table->timestamp('expiry_time')->nullable();
    $table->string('image')->nullable();
    $table->enum('status', ['available', 'reserved', 'claimed', 'expired'])->default('available');
    $table->timestamps();
});
```

The vendor_id employs cascading delete to ensure that all listings are removed when a vendor account gets deleted. The category_id can be null, using nullOnDelete() to keep listings intact even when a category is deleted. The status enum manages the visibility of listings during the entire reservation process.

4.2 Routes Configuration

Routes in FoodSaver are categorized into four groups located in routes/web.php:

- Public routes: The landing page (/) is accessible to all visitors without authentication.
- Auth routes (guest middleware): The /register and /login routes are only accessible to unauthenticated users, preventing logged-in users from accessing these forms.
- Vendor routes (auth + role:vendor middleware): All vendor routes are prefixed with /vendor and named with the vendor namespace. A resource route in Laravel creates the complete collection of CRUD routes (index, create, store, show, edit, update, destroy) for the listings resource.
- Buyer routes (auth + role:buyer middleware): Prefixed with /buyer, these routes allow buyers to browse listings, view listing details, manage their reservations, make new reservations, and cancel existing ones.
- Admin routes (auth + role:admin middleware): Prefixed with /admin, these routes provide administrators with access to a management dashboard, user directory, and listings with deletion options.

// Vendor — resource route with role middleware

```
Route::middleware(['auth', 'role:vendor'])->prefix('vendor')->name('vendor.')->group(function () {  
    Route::resource('listings', VendorController::class)->names(['...']);  
});
```

// Buyer — manual routes with role middleware

```
Route::middleware(['auth', 'role:buyer'])->prefix('buyer')->name('buyer.')->group(function () {  
    Route::get('/listings', [BuyerController::class, 'index'])->name('listings.index');  
    Route::post('/reserve/{listing}', [ReservationController::class, 'store'])->name('reserve');  
    Route::patch('/reservations/{id}/cancel', [ReservationController::class, 'cancel']);  
});
```

4.3 Controllers & CRUD Logic

The VendorController handles all CRUD operations for food listings. It is initialized with auth and role:vendor middleware in its constructor to enforce access control at the controller level.

Create (store method): The store() method checks the incoming request data for validity using Laravel's integrated validation. Required fields include title (string, max 255), quantity (integer, min 1), price (numeric, min 0), and pickup location (string). Optional fields include description, category_id (must exist in categories table), expiry_time (must be a future date), and image (must be an image file, max 2MB). When an image is uploaded, it gets saved in the public/listings folder using Laravel Storage. The authenticated user's ID is assigned to the vendor_id prior to invoking FoodListing::create(\$data).

```
public function store(Request $request)
{
    $data = $request->validate([
        'title'          => 'required|string|max:255',
        'description'     => 'nullable|string',
        'quantity'        => 'required|integer|min:1',
        'price'           => 'required|numeric|min:0',
        'category_id'     => 'nullable|exists:categories,id',
        'pickup_location' => 'required|string|max:500',
        'expiry_time'     => 'nullable|date|after:now',
        'image'           => 'nullable|image|max:2048',
    ]);
    if ($request->hasFile('image')) {
        $data['image'] = $request->file('image')->store('listings', 'public');
    }
    $data['vendor_id'] = Auth::id();
    FoodListing::create($data);
    return redirect()->route('vendor.listings.index')->with('success', 'Listing created.');
```

Read (index/show methods): The `index()` method retrieves only the authenticated vendor's listings, eager-loading the related category to avoid N+1 queries. Results are paginated at 10 per page and ordered by newest first. The `show()` method additionally eager-loads reservations with their buyer details for a complete listing overview.

Update (update method): Similar validation criteria are applicable. When a new image is uploaded, the previous image is removed from storage using `Storage::disk('public')->delete()` prior to saving the new one. The `update()` function is restricted to the vendor's own listings by using `Auth::user()->foodListings()->findOrFail($id)` to avoid unauthorized modifications.

Delete (destroy method): The `destroy()` method obtains the listing specific to the authenticated vendor, removes the related image from storage if available, and subsequently deletes the database entry. Database cascade rules will automatically remove any linked reservations.

4.4 User Authentication & Security

The FoodSaver authentication system utilizes a custom implementation based on standard Laravel session-based auth, avoiding third-party packages to retain complete authority over the registration, login, and redirection processes.

The `RegisterController` manages the onboarding process by verifying six specific fields: name, a unique email, a password (which must at least contain eight characters and confirmation), a role restricted to either 'vendor' or 'buyer', and optional phone and address details. Once validation passes, the user is persisted to the database with a hashed password generated by `Hash::make()`. Upon creation, the application immediately authenticates the new user via `Auth::login($user)` and redirects them to their respective dashboard.

For returning users, the `LoginController` checks the provided credentials and employs `Auth::attempt()` to confirm their identity. After a successful login, the controller creates a new session ID to enhance security against session fixation attacks, leading the user to a dashboard customized to their specific role.

Role-Based Access Control (RBAC) is implemented via a custom RoleMiddleware class that acts as a gatekeeper for protected routes. This middleware examines the role parameter linked to a route definition like 'role:vendor' and compares it with the authenticated user's role field in the database. When the user's role fails to meet the criteria, the system denies access and issues a 403 Unauthorized response.

4.5 Views & Blade Template Engine

All views in FoodSaver are based on a single main layout specified in resources/views/layouts/app.blade.php. This structure fetches shared assets: Google Fonts (DM Serif Display + Outfit), Bootstrap 5 CSS, Bootstrap Icons, and personalized CSS design tokens in the <head>, and includes Bootstrap JS at the bottom of the body.

Child views extend the layout using @extends('layouts.app') and inject their content into named sections using @section('title') for the page title and @section('content') for the main body. This ensures a consistent header, navbar, and footer across all pages with no code duplication.

The navbar dynamically adjusts its links based on the authentication state using @auth, @guest, and @if directives. Links for vendors (Add Listing, My Listings) and buyers (Browse Food, My Reservations) are displayed conditionally depending on the role of the authenticated user.

```
{{-- Navbar conditional rendering based on role --}}

@auth

    @if(auth()->user()->isVendor())

        <a href="{{ route('vendor.listings.index') }}">My Listings</a>

        <a href="{{ route('vendor.listings.create') }}">Add Listing</a>

    @elseif(auth()->user()->isBuyer())

        <a href="{{ route('buyer.listings.index') }}">Browse Food</a>

        <a href="{{ route('buyer.reservations') }}">My Reservations</a>

    @endif

@endauth
```

The buyer listing feed uses @foreach to iterate over paginated listings and @forelse to show a friendly 'No listings found' message when the query returns no results. Food listing cards display the

item's image (with a fallback placeholder), title, vendor name, price, pickup location, expiry countdown, and category badge. The `@if` directive is used to conditionally show the reservation button only when a listing is still available.

5.0 USER INTERFACE DESIGN

5.1 Use of Media (20 marks)

The FoodSaver platform incorporates a mix of static and dynamic media to create an intuitive and visually engaging environment aimed at reducing food waste:

- **Dynamic Food Images:** Managed natively via Laravel Storage, uploaded food images are stored in the public/listings directory. These graphics provide buyers with a transparent, realistic view of the surplus food items before reservation.
- **Categorization Badges:** Utilizing Bootstrap 5 contextual badge classes (.badge), listings dynamically pull standard classification data from the categories table to display color-coded tags (e.g., Bakery, Catering, Groceries).
- **Real-time Status Indicators:** Visual indicators reflect the state of the status enum column (available, reserved, claimed, expired) utilizing highly scannable color semantics (e.g., green for available, yellow for reserved) to instantly update the user.
- **Interactive System Icons:** The interface incorporates Bootstrap Icons (bi-geo-alt for pickup locations, bi-clock for expiry time count-downs, and bi-person for profile management) to minimize cognitive load.

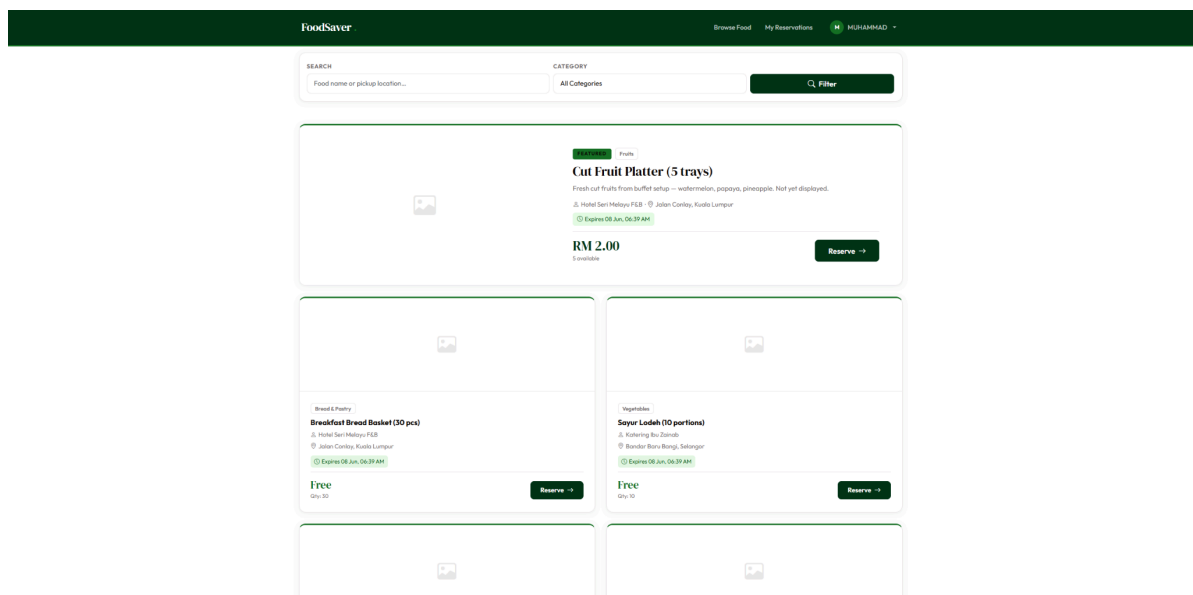


Image 5.1 - Buyer (Browse and Reserve Food)

5.2 Design, Colour Scheme & Layout (20 marks)

The user interface design of FoodSaver relies heavily on color psychology and modern UI frameworks to align with its core mission of sustainability and community welfare:

- **Colour Palette:** Built around an **Eco-Green** primary palette (#2E7D32 / #4CAF50) to symbolize environmental preservation, paired with **Warm Harvest Accents** (#FF9800) to evoke appetite and represent food. Neutrals consist of clean off-whites and dark slate text for balanced readability.
- **Typography Choices:** The application imports Google Fonts within the main app.blade.php layout. It uses **DM Serif Display** for distinct headings to give an elegant, grounded feel, and **Outfit** as the clean, highly legible sans-serif typeface for regular body copy and forms.
- **Grid Structure & Hierarchy:** Built entirely on **Bootstrap 5**, the platform utilizes a mobile-first responsive grid system (.container, .row, .col-md-4). Content hierarchy is cleanly segmented using strict spacing utilities (.mb-4, .py-5) and distinct font weights to guide the user's eye from critical listing details down to call-to-action buttons.

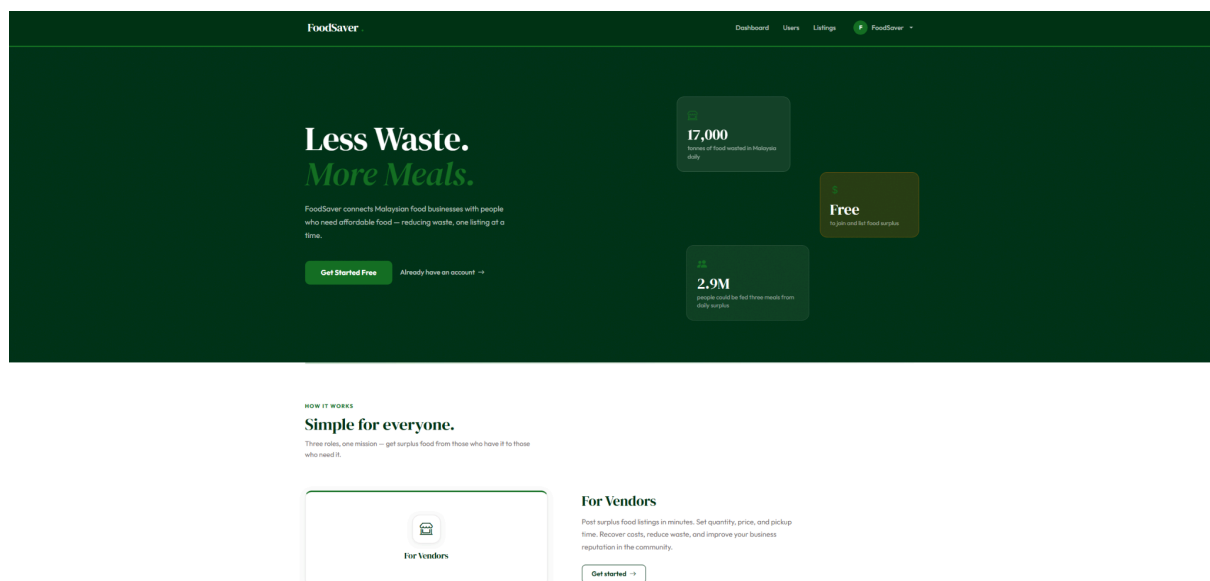


Image 5.2.1 - Guest (Landing Page)

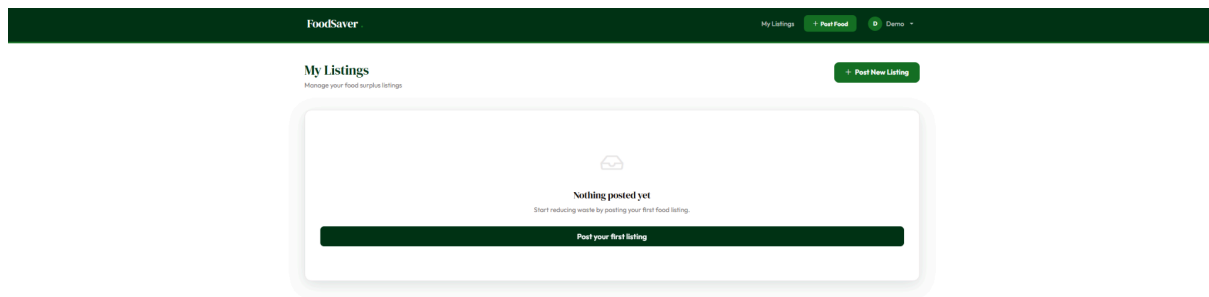


Image 5.2.2 - Vendor (Dashboard)

5.3 Navigation & Links (10 marks)

The navigation layout is controlled globally via resources/views/layouts/app.blade.php. It utilizes Laravel Blade directives (@auth, @guest, @if) to dynamically change the UI according to the user's session state:

- **Guest Mode:** Displays the application logo, "Home", "Browse Food", along with clear call-to-action buttons for "Login" and "Register".
- **Vendor Mode:** Displays vendor-specific management actions such as "**My Listings**" (linking to vendor.listings.index) and "**Add Listing**" (linking to vendor.listings.create).
- **Buyer Mode:** Renders consumer-centric shortcuts consisting of "**Browse Food**" (linking to buyer.listings.index) and "**My Reservations**" (linking to buyer.reservations).
- **Fallback Redirects:** Protected routes are guarded by RoleMiddleware. If an unauthorized user or guest attempts to access a /vendor/* or /buyer/* prefix path, the application blocks the request execution and returns an HTTP 403 Unauthorized fallback message or forces a redirect back to the login screen.



Image 5.3.1 - Guest Navigation

Image 5.3.2 -Vendor Navigation

Image 5.3.3 -Buyer Navigation

5.4 UI Walkthrough (Page by Page)

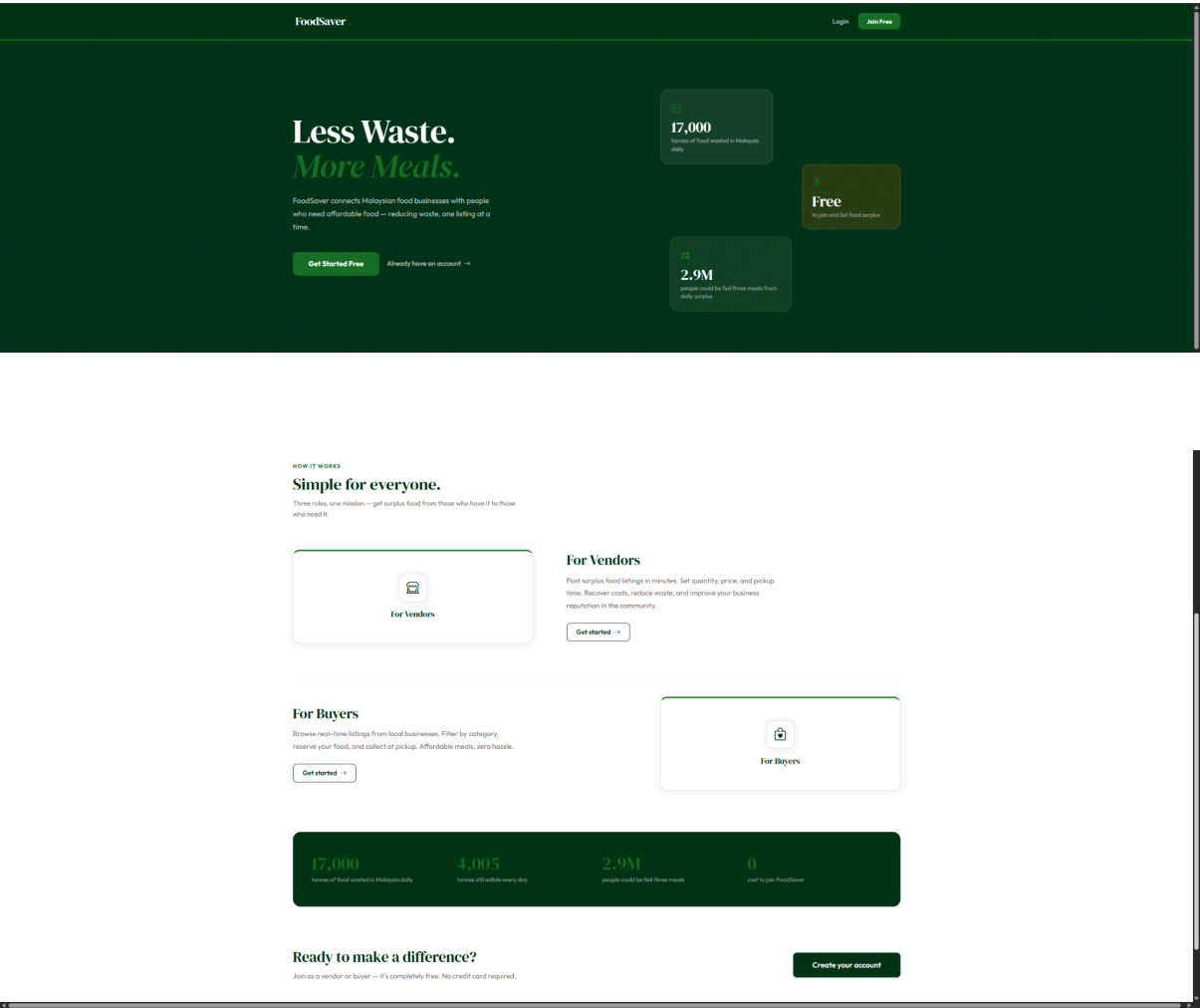


Image 5.4.1 -Landing Page

Features a welcoming hero section built with a bold headline framing the app's mission against food waste. It includes direct call-to-action links to register or browse available surplus food immediately.

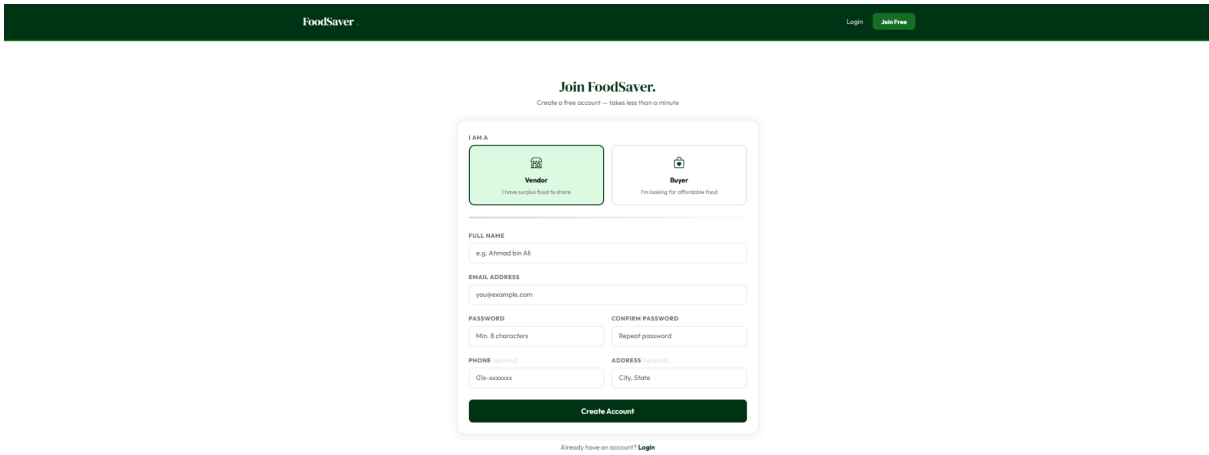


Image 5.4.2 -Registration and Login Page

Clean, centered dual-card layouts built with interactive validation states. The registration form contains a mandatory dropdown selection field for designating account roles (vendor or buyer).

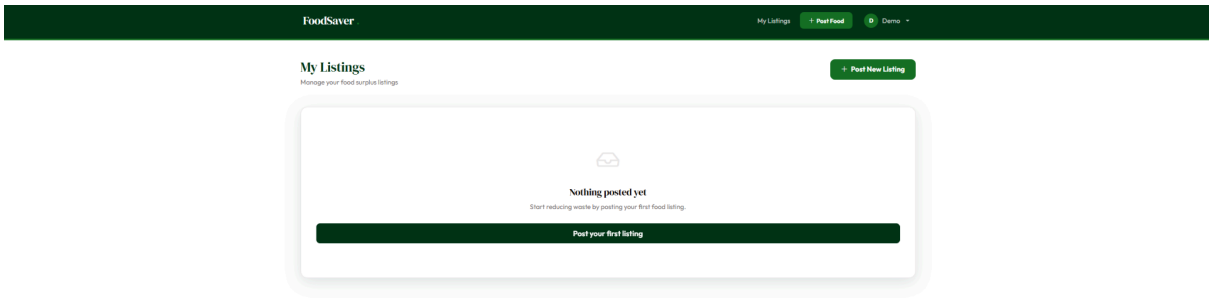


Image 5.4.3 -Vendor Dashboard

A high-level control panel summarizing the authenticated vendor's inventory. It displays total listings posted, items reserved, and clear action items to create new entries.

FoodSaver

My ListingsPost FoodD Demo

Post a Listing

Share your surplus food with the community

FOOD ITEM TITLE *

e.g. Leftover Nasi Lemak (10 packs)

DESCRIPTION optional

Ingredients, condition, packaging details...

QUANTITY *

1

PRICE (RM) - 0 = FREE *

0

CATEGORY

None

PICKUP LOCATION *

e.g. Lot 5, Jalan Ampang, KL

EXPIRY DATE & TIME optional

dd-----yy-- --:--

PHOTO optional

Choose FileNo file chosen

JPG, PNG or WebP. Max 2MB

Post Listing

Cancel

Image 5.4.4 -Post New Listing

A comprehensive form mapped strictly to the food_listings schema fields. It includes input fields for title, category selection, price, quantity, an interactive date-time picker for expiration times, and a file upload element for item images.

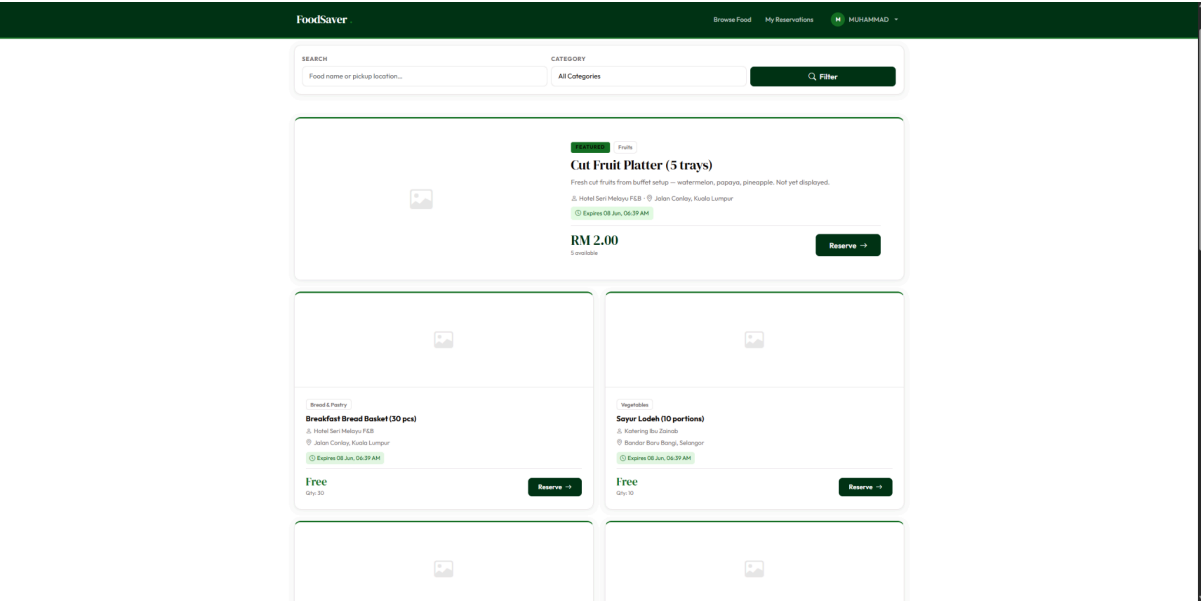


Image 5.4.5 -Buyer Feed Page

An active marketplace interface utilizing a dynamic @forelse loop. It displays all food listings marked as available via interactive bootstrap cards complete with location descriptions, expiration countdowns, and quick-filter tools.

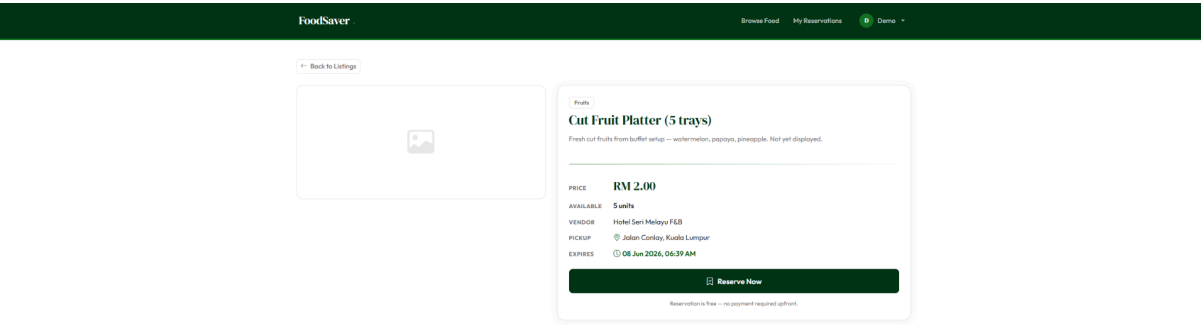
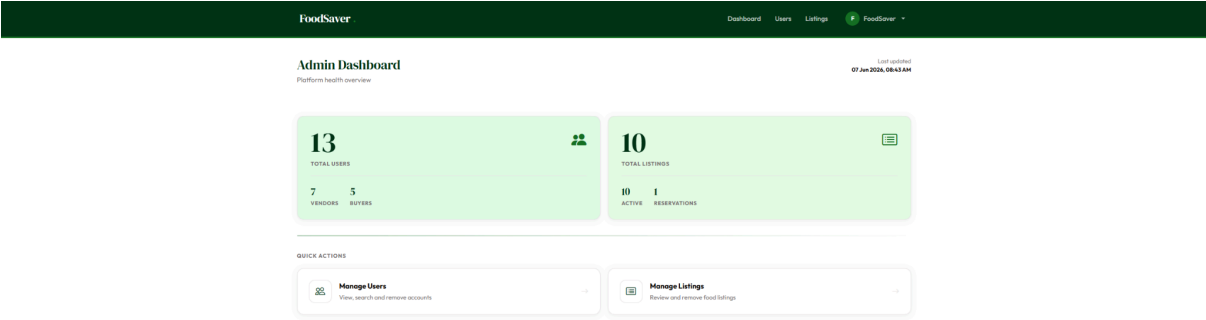


Image 5.4.6 -Reservation Page

A secure confirmation summary view showing the item's details, pickup conditions, and a primary confirmation action that submits a secure POST /reserve/{listing} request to lock the item down.



FoodSaver

DashboardUsersListingsFoodSaver

Manage Users

← Dashboard

All registered accounts on the platform

#	NAME	EMAIL	ROLE	PHONE	JOINED	ACTION
10	G qafinaa	afyrajitnandagmail.com	BUYER	0164054996	06 Jun 2026	Remove
10	H MUHAMMAD FARHIN BIN JOHAR	nomafrhincv24@gmail.com	BUYER	0197790971	05 Jun 2026	Remove
11	H nadi birli esmadi	nadonr123@gmail.com	VENDOR	0123456789	05 Jun 2026	Remove
10	K Naadli	nandagmail.com	BUYER	—	30 May 2026	Remove
9	T Test User	test@foodsaver.my	BUYER	011234567	28 May 2026	Remove
6	H Hotel Seri Malaysia PEB	hotelseri@sg.foodsaver.my	VENDOR	0123005005	28 May 2026	Remove
7	K Kahering Ibu Zainab	kaheringibu@foodsaver.my	VENDOR	0123004004	28 May 2026	Remove
6	K Kedai Kak Mira	kedaikek@foodsaver.my	VENDOR	0123003003	28 May 2026	Remove
6	K Ruti Canal Pak Man	ruticanal@foodsaver.my	VENDOR	0123002002	28 May 2026	Remove
6	K Nasi Lemak Wangi	nasilemak@foodsaver.my	VENDOR	0123001001	28 May 2026	Remove
3	D Demo Buyer	buyer@foodsaver.my	BUYER	0198765432	28 May 2026	Remove
2	D Demo Vendor	vendor@foodsaver.my	VENDOR	0123456789	28 May 2026	Remove
1	F FoodSaver Admin	admin@foodsaver.my	ADMIN	—	28 May 2026	Protected

FoodSaver

Dashboard

Users

Listings

FoodSaver

Manage Listings

← Dashboard

All food listings posted on the platform

#	FOOD ITEM	VENDOR	CATEGORY	PRICE	STATUS	POSTED	ACTION
10	Cut Fruit Platter (3 trays)	Hohel Seri Malaysia F&B	Fruits	RM 2.00	AVAILABLE	28 May 2026	Remove
9	Breakfast Bread Basket (30 pcs)	Hohel Seri Malaysia F&B	Bread & Pastry	Free	AVAILABLE	28 May 2026	Remove
8	Sayer Lodeh (10 portions)	Kichering Bu Zainab	Vegetables	Free	AVAILABLE	28 May 2026	Remove
7	Ayam Masak Merah (15 portions)	Kichering Bu Zainab	Meat & Poultry	Free	AVAILABLE	28 May 2026	Remove
6	Cupcakes Assorted (12 pcs)	Kadai Kak Mira	Desserts	RM 1.00	AVAILABLE	28 May 2026	Remove
5	Kak Bertik (1 loaf)	Kadai Kak Mira	Desserts	RM 5.00	AVAILABLE	28 May 2026	Remove
4	Tostel & Chutney (8 sets)	Ruti Canal Pak Man	Bread & Pastry	RM 1.00	AVAILABLE	28 May 2026	Remove
3	Roti Canal Plain (20 pieces)	Ruti Canal Pak Man	Bread & Pastry	RM 0.50	AVAILABLE	28 May 2026	Remove
2	Mee Goreng Mamak (5 portions)	Nasi Lemak Wangi	Rice & Noodles	Free	AVAILABLE	28 May 2026	Remove
1	Nasi Lemak Bungkus (10 packs)	Nasi Lemak Wangi	Rice & Noodles	RM 1.50	AVAILABLE	28 May 2026	Remove

Image 5.4.7 - Admin Panel

A restricted back-end overview rendering a systemic management directory with clean data tables tracking total users, active listings, and administrative tools to delete non-compliant entries.

6.0 SYSTEM TESTING

6.1 Test Cases Table

Test No.	Feature Tested	Test Input	Expected Result	Actual Result	Status
T-01	User Registration	Valid details, role chosen as 'Vendor'	Account created; hashed password saved; redirected to Vendor Dashboard.	Account created smoothly; redirected to Vendor Dashboard.	PASS
T-02	User Authentication	Correct credentials via Login page	Application initializes session ID; safely logs user in based on database role.	Authenticated successfully; redirected to role dashboard.	PASS
T-03	Create Food Listing	Valid inputs, category selected, image uploaded	Form validates; image stored in public/listings; DB inserts new row.	Item successfully created and visible on vendor list.	PASS
T-04	Form Validation (Expiry)	Setting an expiration date in the past	Validation error triggered via after:now rule; form blocks submission.	Screen reloaded showing "The expiry time must be a date after now."	PASS
T-05	Update Food Listing	Editing an item, uploading a replacement image	Prev image deleted via Storage::disk()->delete(); DB row updates.	Listing updated; outdated image file successfully replaced.	PASS
T-06	Delete Food Listing	Deleting an existing active item	active itemDatabase entry removed; linked reservations drop automatically via cascade.	Listing deleted; database integrity preserved completely.	PASS

T-07	Browse Food Listings	Buyer accesses the browse feed layout	System queries available food items, eager-loading related categories.	Items displayed correctly with no N+1 query lag.	PASS
T-08	Reserve Food Item	Buyer clicks reserve on an available item	ReservationController inserts a row; item status shifts to reserved.	Reservation recorded; listing status updated to 'reserved'.	PASS
T-09	Duplicate Click Prevention	Rapid clicking on "Reserve" button	whereIn('status', ['pending', 'confirmed'])->exists() flags request; blocks it.	Second request caught; redirected back with a flash error.	PASS
T-10	Unauthorised Access	Buyer trying to access /vendor/listings url	RoleMiddleware intercepts request; matches role value; blocks access.	Server returns a clean HTTP 403 Unauthorized restriction.	PASS
T-11	Mobile Responsiveness	Resizing browser viewport to mobile dimensions	Grid layout stacks vertically using Bootstrap flex properties; navbar collapses.	Elements wrap cleanly; touch points remain functional.	PASS

6.2 Deployment Verification

Live Application URL: <https://food-management-production.up.railway.app/>

GitHub Repository Source:

<https://github.com/hwhwbw-root/food-management/blob/main/README.md>

Production Configuration: Deployment was verified directly on the Railway Cloud platform. The live environment connects seamlessly to a cloud-hosted MySQL database instance. Critical production parameters configured within Railway's static environment variable engine include:

- APP_ENV=production & APP_DEBUG=false to ensure user security.

- APP_KEY matching Laravel standards for session encryption.
- Dynamic DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, and DB_PASSWORD variables mapping to the production schema instance.
- FILESYSTEM_DISK=public configured alongside executed php artisan storage:link structures to serve user-uploaded assets flawlessly on production paths.

7.0 CHALLENGES AND SOLUTIONS

During the development of FoodSaver, our team faced several issues that tested our problem-solving skills within the 3 weeks sprint.

7.1 Technical Challenges

1. **Role-Based Access Control Implementation:** One of the early challenges faced during development was to restrict buyers from accessing vendor-specific routes. To ensure that each role could only access their respective dashboard and features require thorough middleware configuration in Laravel. It was then resolved by implementing a custom middleware that checks the user's role value upon authentication and redirects unauthorised access attempts accordingly.
2. **Foreign Key Constraint Errors:** In the early migration phase, there were problems trying to drop or modify tables that were linked via foreign keys. We learned how to use `Schema::disableForeignKeyConstraints()` in our migration files to handle database changes safely.
3. **Deployment Environment Differences:** Deploying the application from a local XAMPP environment to Railway introduced several configuration differences, including variable mismatch, database connection settings, and storage path issues that did not exist locally. We resolved this by updating the `.env` file for the production environment, configuring the Railway MySQL database credentials, and rerunning migrations on the live server to ensure the database structure matched the local version.
4. **Duplicate Reservation:** During testing, we discovered that a buyer could actually place multiple reservations for the same listing by rapidly clicking the Reserve button before the page redirected, causing integrity issues where a listing was marked as 'Reserved' but multiple reservation records existed in the database. To solve this, the `ReservationController's store()` method was updated with an explicit database check using `Reservation::where('listing_id', $id)->where('buyer_id', Auth::id())->whereIn('status', ['pending', 'confirmed']->exists()` before creating a new reservation. If a matching reservation already exists, the user will be redirected back with an error flash message.

7.2 Team & Time Constraints

1. **Accelerated Development Timeline:** The 3 weeks period was our main limitation. To manage this, we implemented an Agile approach, focusing on the 'must have' features (Authentication, CRUD, and Browsing) instead of the 'nice to have' features.
2. **Managing Diverse Technical Skill Sets:** Our team consisted of members with varying levels of proficiency in Laravel and PHP development. Those who were less experienced in backend coding encountered a challenging learning curve while working with MVC architecture and database migrations. To address this technical gap, we employed pair programming and conducted internal code reviews. Teammates with more experience guided others on Laravel syntax and logic, and teammates with less technical skills focused on documentation and user interface design, enabling the team to meet the tight deadline while promoting a culture of collaborative learning and professional development.

8.0 CONCLUSION

8.1 Summary of Achievements

The FoodSaver project has successfully delivered a robust, centralized web application built on the Laravel MVC framework tailored specifically toward mitigating the compounding crisis of edible food wastage in Malaysia. The development group successfully materialized a multi-role user architecture, establishing clear data boundaries via a custom RoleMiddleware layer separating Vendors and Buyers.

Vendors can seamlessly manage inventory lifecycle events via a complete CRUD interface, while buyers benefit from real-time visibility over surplus food items, shifting consumer habits away from unstructured, unreliable communication channels. By facilitating automated reservations and managing data structures through an optimized MySQL schema, FoodSaver presents a reliable framework supporting community welfare and reducing wasteful consumption (*Israf*) in accordance with Islamic values. The system stands fully tested, verified, and successfully deployed live on Railway.

8.2 Future Improvements

To expand the operational efficiency and community impact of the FoodSaver platform, several key upgrades are proposed for future development iterations:

- **Integrated Secure Payment Gateway:** Implementation of third-party payment APIs (such as ToyyibPay or Stripe) to handle micro-transactions directly on the app, allowing vendors to sell discounted items seamlessly.
- **GPS-Based Proximity Filtering:** Integrating geolocation services and Google Maps API to auto-detect a buyer's current location and sort listings based on real-time distance parameters.
- **Real-Time In-App Chat System:** Introducing a WebSockets-driven chat feature utilizing Laravel Reverb or Pusher to establish a direct communication link between vendors and buyers for smooth pickup coordination.
- **Cross-Platform Mobile Application:** Developing native or hybrid mobile apps using frameworks like Flutter or React Native to supply push notifications instantly whenever nearby vendors create new listings.

REFERENCES

Hani, A. (2022, February 15). Malaysia throws away 17,000 tonnes of food daily. *The Malaysian Reserve*.

<https://themalaysianreserve.com/2022/02/15/malaysia-throws-away-17000-tonnes-of-food-daily/>

Laravel LLC. (2023). *Laravel 10.x — The PHP framework for web artisans*.

<https://laravel.com/docs/10.x>

Laravel LLC. (2023). *Eloquent ORM: Getting started*. <https://laravel.com/docs/10.x/eloquent>

Laravel LLC. (2023). *Laravel Blade templates*. <https://laravel.com/docs/10.x/blade>

Laravel LLC. (2023). *Authentication*. <https://laravel.com/docs/10.x/authentication>

Bootstrap Team. (2023). *Bootstrap 5: The world's most popular CSS framework*.

<https://getbootstrap.com/docs/5.3/>

Otwell, T. (2023, February 14). *Laravel 10 released*. Laravel News.

<https://laravel-news.com/laravel-10>

Railway. (2024). *Railway — Infrastructure, instantly*. <https://railway.app>